

Una misma tarea. Cuatro formas de programar.

Encender un LED y transmitir datos por Serial, resolviendo necesidades distintas.

`delay()`

`millis()`

bajo nivel

FreeRTOS

No compiten: cada enfoque responde a un contexto.

delay(): una secuencia fácil de seguir

El programa ejecuta una acción, espera y continúa con la siguiente.

Útil para

Demos rápidas, pruebas de hardware y secuencias simples.

Su calidad

Código directo, legible y muy fácil de explicar.

A cambio de

Durante la espera, la lógica principal no avanza.

Idea mental

Hacer → esperar → hacer → esperar.

MISMA TAREA

```
LED = HIGH;  
delay(500);  
LED = LOW;  
delay(500);  
Serial.println("Hello!");
```

millis(): revisar el tiempo sin detener el loop

Cada actividad avanza cuando se cumple su intervalo; entre tanto, el loop sigue disponible.

Útil para

Varias tareas periódicas, botones y sensores sencillos.

Su calidad

No bloquea y conserva un modelo de ejecución simple.

A cambio de

Los estados y tiempos deben administrarse explícitamente.

Idea mental

¿Ya pasó el intervalo? Si sí, actualiza.

● MISMA TAREA

```
now = millis();
```

```
if (now - ledTime >= 500)  
    toggleLed();
```

```
if (now - serialTime >= 1000)  
    sendData();
```

Registros e interrupciones: control cercano al hardware

GPIO, timers y UART se configuran directamente para obtener comportamiento preciso y predecible.

Útil para

Drivers, restricciones fuertes y temporización precisa.

Su calidad

Máximo control sobre periféricos, memoria y tiempos.

PIC

Registros como TRIS, LAT y módulos Timer/UART.

STM32

RCC, GPIO, TIM, USART y NVIC; directo o mediante HAL/LL.

MISMA TAREA

Timer IR cada 500 ms
→ alternar GPIO

Contador / Timer cada 1000 ms
→ transmitir por UART

PIC: TRIS · LAT
STM32: GPIO · HAL

FreeRTOS: separar responsabilidades en tareas

El scheduler coordina tareas independientes con periodos, prioridades y mecanismos de comunicación.

Útil para

Sistemas con múltiples funciones concurrentes y crecimiento futuro.

Su calidad

Organiza cada responsabilidad como una tarea clara.

A cambio de

Requiere entender prioridades, stacks y sincronización.

Idea mental

LED Task + Serial Task, coordinadas por el scheduler.

● MISMA TAREA

```
LED Task  
toggleLed();  
vTaskDelay(500 ms);
```

```
Serial Task  
sendData();  
vTaskDelay(1000 ms);
```

Scheduler → decide qué tarea corre

La arquitectura correcta depende de la tarea, no de una jerarquía universal.